LLM Inference for Bozos, Part 1

LLMS, C++, PROJECTS

APRIL 14, 2026

First post in a series where I explore the process of writing a high-performance LLM inference engine from scratch. This post will go over the fundamental theory behind LLMs to help readers gain intuition about the flow of information through these models.

Prologue

Large language models (LLMs) have very quickly transitioned from niche research to a part of every day life. When I started at my current job in August, the company had just begun giving people Claude Code budgets and setting up internal LLM tooling. Within a few months, everyone was running any non-trivial task through Claude before anything else.

The more I talk to friends and family across various industries the more everyone reflects on pre-AI work as some sort of archaic period. These tools span every domain now: there's diffusion-powered image generation in Photoshop's harmonize[1] tool and 2D -> 3D rotation feature[2], a slew of LLM coding assistants[3,4,5], or the increasingly popular full blown LLM personal assistant[6].

THE ROAD AHEAD

This series of posts is going to build up a state of the art LLM inference engine from scratch. This post is going to go into the core theory behind LLMs. I am working with the assumption that you are proficient with general programming principles and have some high-school level math background, but little to no knowledge about the internals of LLMs. This post is dedicated to *inference*, I will go over some training details as they pertain to the overall architecture, but it is not the focus of this post.

This first post covers the fundamental theory and intuition behind LLMs, starting with basic neural networks and working our way towards transformers. The next post will focus on inferring a simple model like Llama 3.2 1B, and future posts will work towards a more modern model like the recently-released Gemma 4. The initial code is plain C++ with minimal dependencies. Future posts will port to the GPU in Metal shading language, targeting performance on par with optimized frameworks like llama.cpp or MLX.

LOCAL LLMS

Everything we write should run on my own laptop. Most similar writeups run their kernels on rented server GPUs, which makes sense in the context of preparing to write datacenter-scale training and inference kernels at large companies. But on top of not wanting to spend money on renting compute, local models are getting good enough to be genuinely useful.

For the unfamiliar, there are two ways you can use LLMs. The overwhelming majority of people will interact with them through some web interface like ChatGPT, Claude, or Gemini. In these cases, the model's weights are sitting in memory in a datacenter filled with server-grade GPUs like B200s. Every prompt you send travels to a server that processes it alongside other users' requests and streams the output back. This means that the model's weights live on the server and your computer just sends messages over to it. Flagship models like Claude are too large to fit on any consumer hardware regardless.

We'll go over what weights mean in the context of machine learning soon, but the easiest way to think about it is they are the coefficients in all of the math that the model is doing to produce an output.

The alternative to this is running a model locally: you download the weights of an open-source model like LLama 3.2 1B[7] and have your computer do all of the work to generate the outputs. This comes with a few major downsides. You are limited to using whatever models have their weights available online (which are less capable than the household names), and you need hardware powerful enough to run that model on. The upside is that the model is yours, it doesn't degrade, doesn't change, and it's free.

A very important element if you're worried about rate limits, model performance degrading, or for whatever reason emotionally attached to your model[8]

Neural Networks

Nearly all modern machine learning is built on neural networks. We care about them because of the Universal Approximation Theorem[9] which says that a neural network with a single layer and with sufficient width appropriate activation function can be used to approximate any continuous function to any desired degree of accuracy. The bet behind LLMs is that by training sufficiently large networks to learn the underlying distribution of language, we are training a general model of the world.

In reality it is heavily debated what LLMs actually "learn". Many argue they are 'stochastic parrots' that convincingly interpolate between training data to spit out believable responses. Others believe that in approximating the underlying distribution of language, models learn the rules of the world that generated it.

THE NEURON

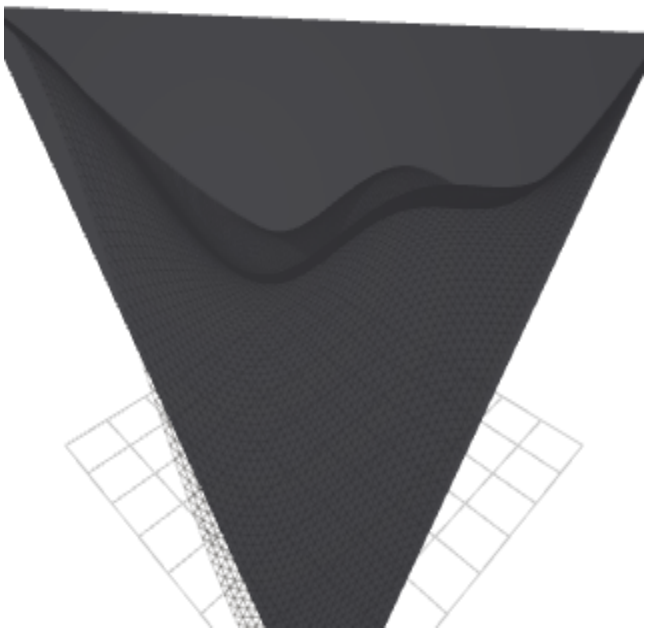
Neural networks are surprisingly simple at their core. A single neuron contains a trained weight vector, $\vec{w} = [w_1, w_2, ..., w_n]^T$, and bias scalar b. Given an input vector $\vec{x} = [x_1, x_2, ..., x_n]^T$, each neuron computes $z = \vec{w}^T \vec{x} + b$. z is then put through a non-linear activation function $g(z)$.

THE LAYER

Individual neurons are organized into layers. A layer is simply a collection of independent neurons that together, project our input data into a different dimension. If we have n features, our input vector $\vec{x}$ lives in $\mathbb{R}^n$. Then for a layer with m neurons, each will have a weight vector $\vec{w} \in \mathbb{R}^n$, which we can express as a weight matrix $W \in \mathbb{R}^{m \times n}$ (simply all the weights stacked on top of each other). We can similarly stack each neuron's bias scalars to form our bias vector $\vec{b}$. Our output vector for the entire layer is now $\vec{z} = W\vec{x} + \vec{b}$. Overall, the layer performs an affine transformation $T(\vec{x}) : \mathbb{R}^n \to \mathbb{R}^m$.

If we stopped here, and chained layers together like this, we wouldn't actually be able to learn much. We would be left with a series of affine transformations on our input data. The product of two affine transformation matrices is also affine, meaning we would simply be doing a series of scales, rotations, shears, and translations to our input vector. No matter how many layers we have, they would all collapse into a single affine transformation and we could only learn affine functions. This is why we introduce activation functions: we pass the affine output through a non-linear function that "folds" the vector space into complex shapes. We are then left with many

linear segments (one per neuron) that are used to approximate any arbitrary smooth multi-dimensioned function.



Visualization shows an example of how with more separating planes, we can better fit the surface of complex distributions. Try changing the number of neurons, or enter a new function to fit.

Three types of layers make up a network. An input layer takes raw data (pixel values, word encoding, sensor readings, etc) and compresses it down to a dense latent representation.

Followed by a series of "hidden" layers (hidden because they do not directly interface with the real-world data, and because they are used to learn higher dimensional relationships) that project their input layers into a higher dimensional space. We are going from a dense lower-dimensional representation of information into a much sparser high-dimensional space. This helps to better isolate features. We have many more neurons to work with that activate under more specific circumstances which creates sparsity in our activations and more unique combinations. So where in lower dimensions, multiple features share the same dimension, in higher dimensions operations have the room to occupy their own dimensions and act on more specialized tracts of neurons.

For a concrete example here, imagine that we are working with a "dictionary" of 50,000 words that the model recognizes. Then every single word is encoded as a unique vector in $\mathbb{R}^{50,000}$. The input layer will take the vector corresponding to some word we feed it, and project it into a continuous latent vector space, e.g. $\mathbb{R}^{4096}$. Every hidden layer will then expand its input from $\mathbb{R}^{4096}$ to e.g. $\mathbb{R}^{16384}$. After it applies its activation function, it will then recompress back to $\mathbb{R}^{4096}$, add it to the input vector, and pass it on to the next layer. Finally, the output layer will take our $\mathbb{R}^{4096}$ vector that has been enriched by the hidden layers, and project it to our original $\mathbb{R}^{50,000}$.

Finally, we have an output layer which takes the vector produced by the final hidden layer and undoes the transformation done by the input layer, projecting it back into our original vector space.

THE NETWORK

A neural network is made up of an input layer, hidden layer, and output layer. Two or more hidden layers makes it a 'deep' neural network. Much of the power of neural networks in modern machine learning comes from chaining layers together in these deep networks. In theory you can learn any underlying distribution in a neural network with a single hidden layer with enough neurons. In practice we see that using a single large layer ends up brute-forcing the distribution, each neuron represents some highly localized feature in what we are trying to represent. In other words, the network's ability to create distinct linear segments grows linearly with width. For a function that is 10x as complex, we need 10x the number of neurons.

This is why depth matters. A single layer acts as some non-linear function, multiple layers chained together act as multiple composed functions. This allows for each layer to learn more abstract concepts rather than relying on a single layer to make the leap from input to output.

One nice way to think about the effects of having multiple layers is through image processing. If we are training a network to recognize faces, the first layer may implicitly learn to detect edges, the second curves and corners, the third features of the face, and a final layer specific faces. If we only had one layer, we would be expecting the single hidden layer to learn the weights to go from pixel values to the abstract concept of a face.

I also want to take some time to talk about what our input data looks like as it goes through a network. Think about processing an image of a face, the same image shifted by a few pixels may look completely different when comparing them pixel by pixel in the input space. The input space is chaotic, vectors carry no semantic meaning. When the raw data passes through the first hidden layer, we know that it projects it into a latent space. These latent spaces are what is learned when we talk about training a neural network. Every layer's weights and biases evolve through the training process to learn various aspects of the training data. Now, in a well-trained network, axes in the latent space represent more meaningful concepts: for example when processing text some axis may represent "negative tone" or "plurality".

One remarkable quality of latent spaces is that they let us perform linear algebra operations on concepts. One of the most famous

examples is that given vectors for different words in this space, we can expect the following to be true: . We can also calculate the distance between vectors to gauge semantic similarity, we expect king and queen to be roughly next to each other in space, separated by the same distance that man and woman are.

$$\vec{v}_{king} - \vec{v}_{man} + \vec{v}_{woman} \approx \vec{v}_{queen}$$

Going back to the image processing example, in our initial data there is no nice way to tell apart dog from cat but we know that there are definitive concepts that make dogs dogs and cats cats. As we go through the hidden layers, the goal of the network is to take our initially meaningless space and "untangle" it till all dog-concepts are clustered together, and all cat-concepts are clustered together.

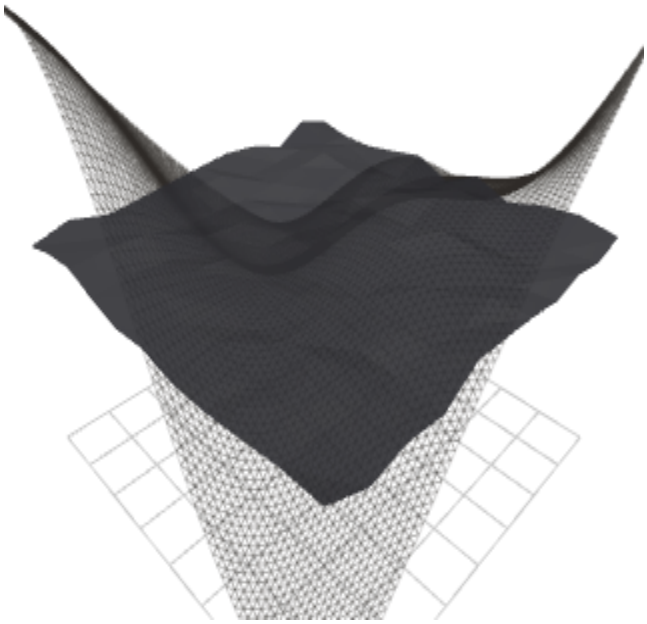
The input space, as we saw previously, has no meaningful geometry. But the data is not random, it lives on some smooth surface (a manifold), there is a fundamental reality that dictates the similarities and differences between concepts. The ultimate goal of all of our layers is to progressively unfold this manifold until we end up in a vector space where all of these concepts are clustered together with meaningful relationships between them. This manifold exists regardless of the space we are in, each layer projects it into a new space that exposes different structure.

TRAINING A NETWORK

Before we can do anything useful with a network, we need to train it. All of the neurons are initialized with completely random weights and biases. Geometrically, our network is effectively randomly stretching and folding our input data, with our inputs ending up at arbitrary positions in the final latent space. To train a network, we feed it many labeled examples. Data will flow through every layer and make a final prediction. We use a mathematical loss function to calculate how far from reality that prediction was.

We define our own loss function for the distribution we are training the data to learn. All the loss function does is find the distance between where the data actually landed in our final latent space, and where it should have landed.

Once the network knows how wrong its guess is, it uses an algorithm called gradient descent to adjust the weights in the right direction. It does this with backpropagation: going backwards through every layer of the network and calculating by how much it needs to tweak its weights and biases. Through thousands of these small adjustments, the network iteratively remodels the mesh formed by all of the piecewise segments represented by the hidden layers to fit the geometry of the underlying function. The visual below shows you how iterative weight updates gradually fit an arbitrary function.



There is a lot of nuance and literature in training that I am glossing over here for the sake of brevity. To learn more about the training process and how neural networks work in general, I highly recommend taking a look at Andrej Karpathy's Neural Networks: Zero to Hero[10]. He goes over building neural networks from scratch, with lots of valuable intuition about what each step in the process is actually doing.

INFERRING A NETWORK

Once training is complete, we can lock the weights and biases of the network in place (given faith that they have actually learned the desired distribution) and use them for inference. Inference means feeding the network new data and letting it predict based on the distribution it learned during training.

Compared to training, inference is much more straightforward. When we talked about the layers of a network we saw that all of the work for a single layer is simply $\vec{z} = W\vec{x} + \vec{b}$ followed by an activation function. Given an input vector, we do our sequence of matrix multiplications, adding biases, and computing activation functions until we reach the

output layer. For discriminative inference (tasks like classification), the output layer draws a hyperplane through the final latent space. Which side of the plane your data lands on is the prediction. For generative inference (things like LLMs or image models), a modern architecture like transformers transforms the input vector into a final vector in the latent space. The output layer considers the distance between that vector and every other known vector in its dictionary. For example given "the monkey ate the ", we might find that the final vector is a 90% match to "banana", a 5% match to "apple" and a 0.0001% match to car. It will append "banana" back to the input, and run the whole sequence again.

One way to look at this is that discriminative machine learning will compute "slices" of the manifold in latent space at training time, and during inference we just have to find where among the slices our data belongs. For generative inference, we take each part of the input and "walk" across the manifold to find the portion that is most likely to follow the input.

Transformers

Given you should now have some intuition around the fundamental concepts of neural networks, let's move onto the basis of LLMs (and most other modern parts of machine learning): the Transformer. Transformers were introduced in Google's seminal 2017 paper Attention is All you Need[11] (Vaswani et al.).

Fun fact, the GPT in ChatGPT stands for "generative pre-trained transformer". Generative in this case meaning it is used to generate text and pre-trained meaning it uses the gradient descent based weight-adjusting scheme we described earlier.

FROM WORDS TO TOKENS

We know that neural networks operate on vectors of numbers, not strings of text. Before we can do any computation we have to convert our raw text into something that the model can work with. One naive approach may be to assign an integer to each unique word in the English language, but this falls apart quickly. We would need to maintain a massive vocabulary to cover every word the model might encounter, and it would have no way to process a word it had never seen before.

LLMs solve this problem with subword tokenization. They use algorithms like byte-pair encoding (BPE) to break text into smaller common reusable pieces called tokens. Under BPE, a word like "newer" may become ["new", "er"] and a longer one like "dehumidifier" may become ["de", "hu", "mid", "if", "ier"]. The model then maintains a fixed vocabulary of tokens to compose any new words, for example LLama 3 has a vocabulary of 128,000 tokens. Each token has some corresponding learned vector, its embedding, that lives in the model's latent space. Earlier we described the input layer

to a neural network, for a transformer this embedding lookup is our input layer. Every time we put text into a model, we split it into tokens, look up each token's embedding vector, and then feed that sequence of vectors to the model. Similarly, the model will spit out vectors that we compare against the full vocabulary to choose which token comes next.

CONTEXT, AMBIGUITY, AND ATTENTION

One of the core problems with vanilla neural networks is that every token has a single fixed position in latent space. No matter what came before it, if the network sees "bark" it will always assign it the exact same vector. This is unlike real language where meaning is highly dependent on the context; bark means something very different depending on whether we're talking about a dog versus a tree. This is the problem solved by *Attention*, introduced by Neural Machine Translation by Jointly Learning to Align and Translate.[12] (Bahdanau et al. 2014).

The original transformer was designed for translation which has a natural divide between input and output. French went in, and English came out. The architecture mirrored this: an encoder would read the input sequence and compress it into a latent representation, and a decoder would take that representation and generate the output in English a token at a time. The encoder is allowed to look at all other tokens in the input regardless of position, because when you are trying to understand a sentence you should have the full context. The decoder is only allowed to look at tokens that came before it, because you can not condition your next token based on tokens you haven't generated yet. LLMs do not use this encoder-decoder break, there is no clean split between input and output in conversation. The prompt and response are treated as being one continuous stream of text. We keep only the decoder and treat input tokens the same as output tokens, only looking at the tokens that came before them.

Attention solves this problem by having tokens in a sequence look at every other token, evaluate their relationship, and adjust their own meaning based on their neighbors. The vector for "bark" will always start off at the same point. After attention, its vector shifts. "bark" in a sentence after "tree" means something very different

from “bark” somewhere in a sentence after “dog” or “peppermint”. Concretely, Transformers implement this by taking every token’s starting vector and multiplying it by three different weight matrices to create three new vectors for each token: the query (Q), the key (K), and the value (V).The query represents what the current token is “looking for”. “bark” may project a vector that says “are there any tree or dog related tokens around me?”. The key represents what the current token is. “bark” may signal “I am a noun”, “I am an object that can be rough”, and “I am a loud action/sound” simultaneously. Finally, the value vector is the pure meaning of the token that gets transmitted to other tokens, its job is to move other vectors to more context aware locations in latent space.

If you are struggling to reason about how the Q & K vectors can represent so many different concepts, it can help to remember that we are working in high dimensional spaces like our example from earlier. $\mathbb{R}^{4096}$ So if axes #5 represents noun-ness, axis #500 represents roughness, and #838 represents being a loud sound, we might see that the key vector for “bark” has spikes on those axes.

What makes this system so powerful is that it decouples what about tokens makes them a good match for other tokens, and what they mean. When we see a sentence like “the fluffy dog barked”, “fluffy” broadcasts “I am a tactile adjective looking for an animal or furniture noun”. But the meaning of fluffy doesn’t matter in that matching process.

The attention computation is very simple, each token in the sequence does a dot product between its query vector, and every other token’s key vectors. Q and K pointing in the same direction implies semantic similarity so their dot products will be high. Unrelated vectors will be orthogonal and their dot product produces zero. Once we have done this computation for a single token, we put all of the vectors through a softmax, a computation that transforms all of them into values summing to 1. This gives us our attention matrix. However, as we increase the dimensionality of these vectors, , these dot products can grow to be very large. When we put the attention vectors through the softmax we could often end up in scenarios where a single score becomes 1.0 and all others are 0.0, killing any ability to learn from multiple tokens at once. d_k

To fix this we scale the dot product. We divide the result by the square root of the dimension of the key vectors . This keeps the values in a stable range, ensuring the softmax produces a smooth distribution of probabilities. This operation is known as scaled dot product attention, expressed as $\sqrt{d_k}$

$$\text{Attention(Q,K,V)} = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Going back to the fluffy dog, if we are computing attention scores for the token "barked" we might see something like:

TOKEN	SCORE	RATIONALE
The	0.03	Not necessary to "barked" for meaning
fluffy	0.08	Might give slight reasoning to type of thing that barked, but not core to meaning.
dog	0.65	Tells "barked" that it is a dog that is barking, the core of what we were looking for.
barked	0.24	Attending to itself, make sure barked preserves some of its baseline meaning.

Remember that this is only the attention computation for the token "barked" we would also do this for every other token in the sentence. For each token, we then get a new context vector with the simple calculation . In other words, it is just a linear combination of each value vector and its corresponding attention score. This should make intuitive sense, if "dog" matters the most to the meaning of "barked" then we should weigh its value contribution to "barked"'s meaning more than "fluffy".We then add the original vector back: . This residual connection ensures we are enriching the original vector, not replacing it.Finally, we do a normalization step to make sure vector magnitudes do not grow boundlessly across layers.

$$\vec{x}_{context} = \sum score_i \times V_i$$

$$\vec{x}_{new} = \vec{x}_{original} + \vec{x}_{context}$$

In practice we would just take all the tokens in our sequence, and create a matrix of all of their Q and K vectors stacked on top of each other to get a query matrix and a key matrix. Multiplying these together gets us our final matrix of attention scores.

POSITIONAL ENCODING

There is one glaring issue with this system. We know tokens "attend" to other tokens to try and gain context, but the simple dot product we use to calculate similarity does not know anything about token positions. The sentences "The dog bit the man" and "The man bit the dog" look mathematically identical to attention.

Before any token is put through any part of the model we stamp it with a positional encoding. Each vector will have a unique signature the model can learn to recognize where in the sequence each token

belongs in. This is as simple as adding on a position vector to each token's original vector: .

$$\vec{x}_{input} = \vec{x}_{token} + \vec{x}_{position}$$

This lets the model learn position-dependent relationships during training. A query like "I need a noun" shouldn't match every noun in the sequence equally. Positional encodings let the model learn that nearby nouns matter more than distant ones. If we have the token "blanket" much earlier in our sequence and then we see "the fluffy dog", we shouldn't attribute fluffy equally to dog and blanket. We should know that fluffy probably doesn't attend to blanket because of how far back it is.

Most modern models generate these embeddings through Rotary Positional Embeddings (RoPE)[13] (Su et al. 2021). What RoPE does is instead of adding a static vector at the start, we rotate all of our vectors during the attention computation. We have some rotation amount , and the token at position gets rotated by . This works perfectly for attention because scores are calculated with a dot product, which depends on the angle between two vectors. With RoPE, two tokens separated by the same distance always have the same relative rotation, regardless of where they appear in the sequence. This gives models a natural bias toward attending to nearby tokens. In short, Attention Score .

θ n $n\theta$

$$\propto \cos(\theta_m - \theta_n)$$

MAKING MEANING FROM CONTEXT

Each token now carries a context-enriched vector: its original embedding plus a weighted blend of its neighbors' meanings, normalized. Each vector now goes through a feed-forward neural network. There is no cross-token communication here, every vector is put through the exact same network independently.

When our context-enriched vector for "barked" comes out of the attention mechanism, it has absorbed the geometric position of "dog" but the vector "barked + dog" doesn't really mean anything, it is just a coordinate in space. It is up to the network to say "this coordinate corresponds to the sound that a small house-pet canine makes". This is the same kind of feed-forward network we covered earlier. Its weights were just trained specifically to interpret the outputs of attention.

Finally, similarly to the end of the attention stage, we do a residual connection followed by a normalization. We add the input to the feed-forward network to the output vector it produces to make sure we do not completely lose the original context we started with, and we normalize that final vector to make sure it lives in a range that following layers recognize.

GENERATING A TOKEN

The sequence we have described above, Attention → Residual & Normalize → Feed-Forward Network → Residual & Normalize, represents a single transformer block/layer. These layers relate to each other the way hidden layers do in a plain neural network. Earlier layers tend to learn things like parts of speech or syntax and grammar, middle layers shift to semantics and longer-ranged relationships, and later layers may represent abstract task-specific concepts. None of this is explicitly wired into the model. We never tell layers to learn grammar or layer 20 to learn semantics. These relationships emerge during training.

There is a field called mechanistic interpretability that focuses on trying to figure out what exactly it is that models learn, and where they learn these things. There is a lot of interesting work being done on things like figuring out what different layers in a pre-trained model do[14] or mapping neuron activations to human emotions.[15]

This sequence is called a post-norm architecture, which is what is described in the original transformer paper. Modern models have moved to a pre-norm architecture in which we have Normalize → Attention → Residual → Feed-Forward Network → Residual. This is largely for gradient stability during training time.

After every token in our input goes through the sequence of transformer blocks, each enriched with progressively deeper context, we're ready to generate new tokens. After the final block, only the last token's vector matters. By now, the vector encodes the full context of every preceding token; relationships, meanings, and structure are compressed to a single point in latent space. We put this single vector through an output layer called the Language Modeling (LM) head. Just like the final output layer in a neural network we described earlier, this LM head simply takes the vector passed into it and maps it against the vocabulary of tokens that the model recognizes. This creates probabilities for every token in the vocabulary, so we can choose one to be our next token. We append the chosen token to the input and run the sequence again. One token is generated per pass, until the model emits a 'stop' token or we reach a length limit.

The LM head is actually interchangeable. We could just as easily replace it with a classification head that will give us a binary yes/no on if the sequence is spam, or a regression head that maps that final vector to a number from 0-100 to tell us how angry a customer was for sentiment analysis.

THE KV CACHE

There is one glaring inefficiency in this computation: we go through the entire transformer doing work for every single token in the input sequence for every single new token we want to generate, but most of

the work we are doing is redundant. This is because during training and inference we apply causal masking. Tokens are only allowed to look at other tokens that came before them. When considering "The fluffy dog barked", "fluffy" is only allowed to look at "The" and "fluffy", it doesn't make sense to give tokens knowledge of the future. This means that we don't care about the queries those tokens broadcast anymore, so we maintain a KV cache.

Every time we generate a new token, we append its K and V vectors to a cache, forming two matrices that grow for every new token we process. When a new token goes through the pipeline, we just have to multiply its Q vector by the entire cached K matrix to generate attention scores and add the weighted V matrix. Instead of multiplying the full Q matrix by the full K matrix every step, we multiply a single query vector against the cached keys, reducing a matrix-matrix computation to a matrix-vector one.

ATTENTION HEADS

One last fundamental topic to modern LLMs is the concept of attention heads. We talked about how attention produces enriched context vectors, but in practice we find that it isn't perfect at capturing relationships. It often ends up producing a muddled average of all of the tokens that were important to the token being attended to. Attention finds multiple tokens equally important but for different reasons and can only produce one blended context vector.

The solution to this is to split attention up into multiple heads that can all learn different concepts. For example, if we are working with a latent space in $\mathbb{R}^{4096}$ then we may have 32 attention heads that each work in $\mathbb{R}^{128}$. This gives us 32 distinct W^Q, W^K, and W^V matrices that all have their own independent set of learned weights for every single transformer layer. The heads naturally learn to specialize; one head may solely try to look for the agent noun that drives some action, another may strictly look for the noun being acted on, another searches for modifiers, etc.

Once we have all of our new context vectors after multi-headed attention, we stack them all together and multiply by a new output matrix W^O. This matrix smooths out the "seams" of where we stitched all of the individual attention matrices to resolve them into a single

vector in the model's latent space. The output projection does three things, it reconciles all of the disjoint head inputs, rotates the concatenated vector back into the standard coordinate system, and lets the network weigh some heads more heavily than others.

Next Steps

That is the full pipeline. Text gets split into tokens, each token looks up its embedding vector, and that sequence of vectors passes through a stack of transformer blocks. Each block runs attention to build context, feeds the result through a network to extract meaning, and adds residual connections to preserve what came before. After the final block, the last token's vector gets projected against the full vocabulary to pick the next token. Repeat until done.

The goal of this post was to build the intuition for what each step of a transformer does and show that none of the individual operations are complicated. Now that we have that understanding, the next post will put it to use. I will show how to look at Llama 3.2 1B's huggingface page, download its weights, and implement each of these operations to run real inference. You can find that at the **next installment of this series**.

REFERENCES

1. https://helpx.adobe.com/photoshop/desktop/repair-retouch/remove-objects-fill-space/blend-subjects-with-harmonize.html

2. https://research.adobe.com/news/new-photoshop-feature-rotate-2d-objects-in-three-dimensions/

3. https://code.claude.com/docs/en/overview

4. https://openai.com/codex/

5. https://cursor.com/

6. https://openclaw.ai/

7. https://huggingface.co/meta-llama/Llama-3.2-1B

8. https://www.reddit.com/r/MyBoyfriendIsAI/

9. https://en.wikipedia.org/wiki/Universal_approximation_theorem

10. https://karpathy.ai/zero-to-hero.html

11. https://arxiv.org/abs/1706.03762

12. https://arxiv.org/pdf/1409.0473

13. https://arxiv.org/pdf/2104.09864

14. https://dnhkng.github.io/posts/rys/

15. https://www.anthropic.com/research/emotion-concepts-function